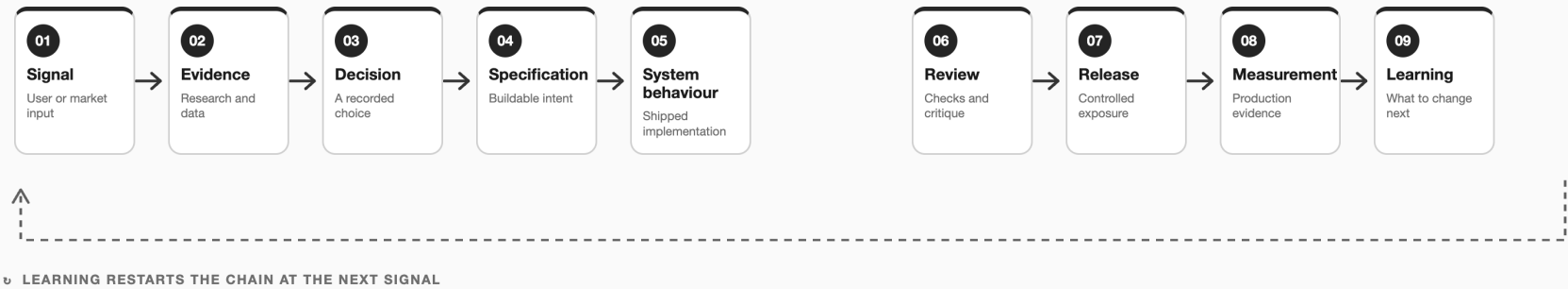


Figure 0.1. Signal-to-learning operating chain

FIGURE 0.1

The signal-to-learning operating chain

Product intent survives only when each transition stays inspectable; learning begins the next loop rather than ending the work.



Each chapter owns one transition; the chain flows to learning, then loops back — it does not end at release.

F0.1

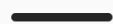
Product intent survives only when each transition remains inspectable; learning begins the next loop rather than ending the work.

Figure 1.1. Participation ladder: ask, inspect, prototype, contribute

FIGURE 1.1

The technical participation climb

Technical confidence grows through observable contribution, not a change in title.



DEEPER PARTICIPATION

04

Contribute: change a team-owned system safely

03

Prototype: create a bounded learning object

02

Inspect: trace behaviour through artifacts

01

Ask: expose states, risks, and ownership

Move only as far as the product question and risk require.

F1.1

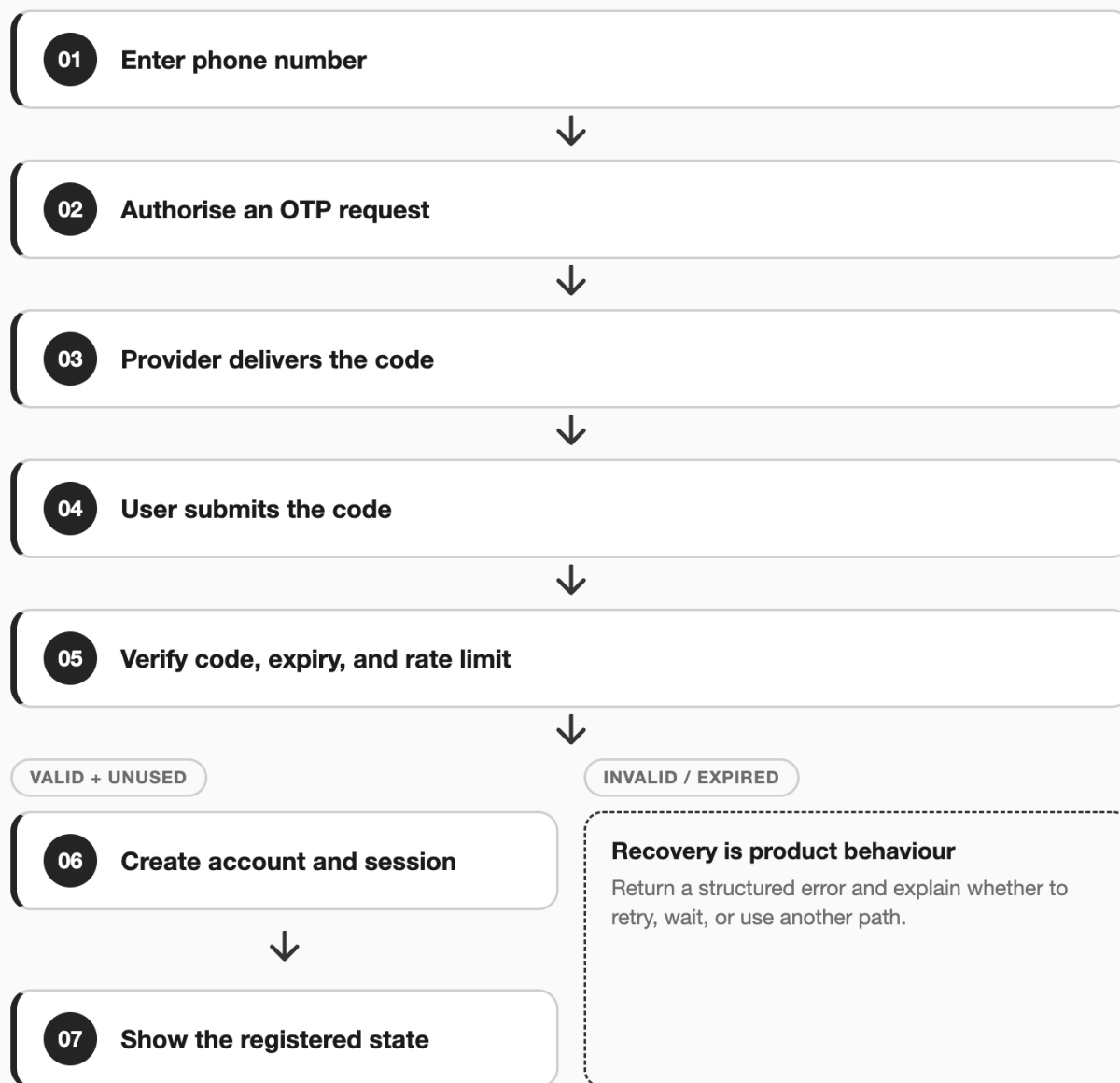
Technical participation progresses through observable work, not status; the appropriate stopping point depends on the product and risk.

Figure 2.1. Registration request/response sequence

FIGURE 2.1

The registration promise

One user action crosses product, identity, provider, storage, and recovery boundaries.



Success and recovery are two outcomes of the same registration promise.

F2.1

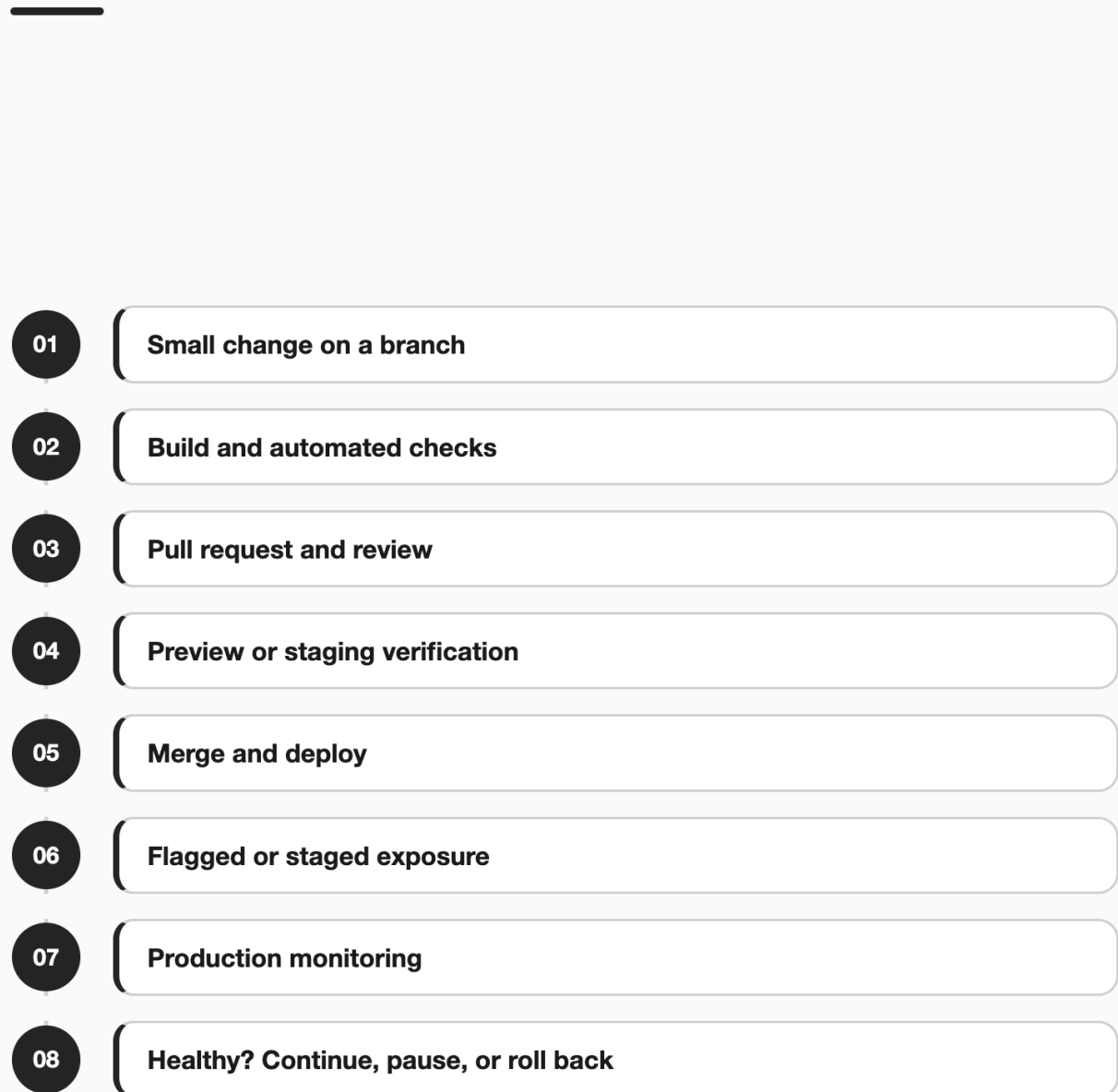
The registration promise crosses multiple actors; success and recovery states both belong in the product conversation.

Figure 3.1. Commit-to-production delivery path

FIGURE 3.1

The delivery confidence path

Integration, deployment, exposure, and learning are separate product decisions.



Shipping confidence accumulates one verified boundary at a time.

F3.1

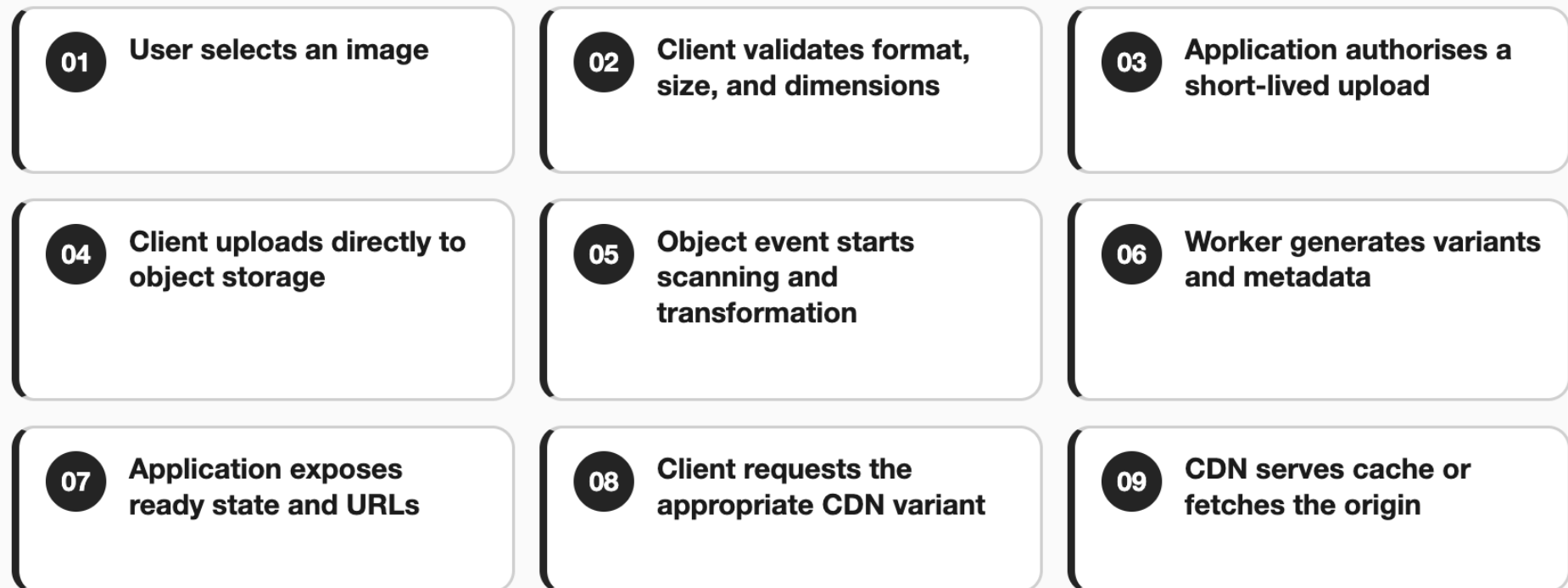
A delivery path separates code integration, deployment, exposure, and the decision to continue.

Figure 4.1. Original asset to responsive delivery

FIGURE 4.1

The image-asset journey

The application authorises the action; specialised systems move and transform the file.



One user action crosses validation, authorisation, transfer, processing, metadata, and delivery boundaries.

F4.1

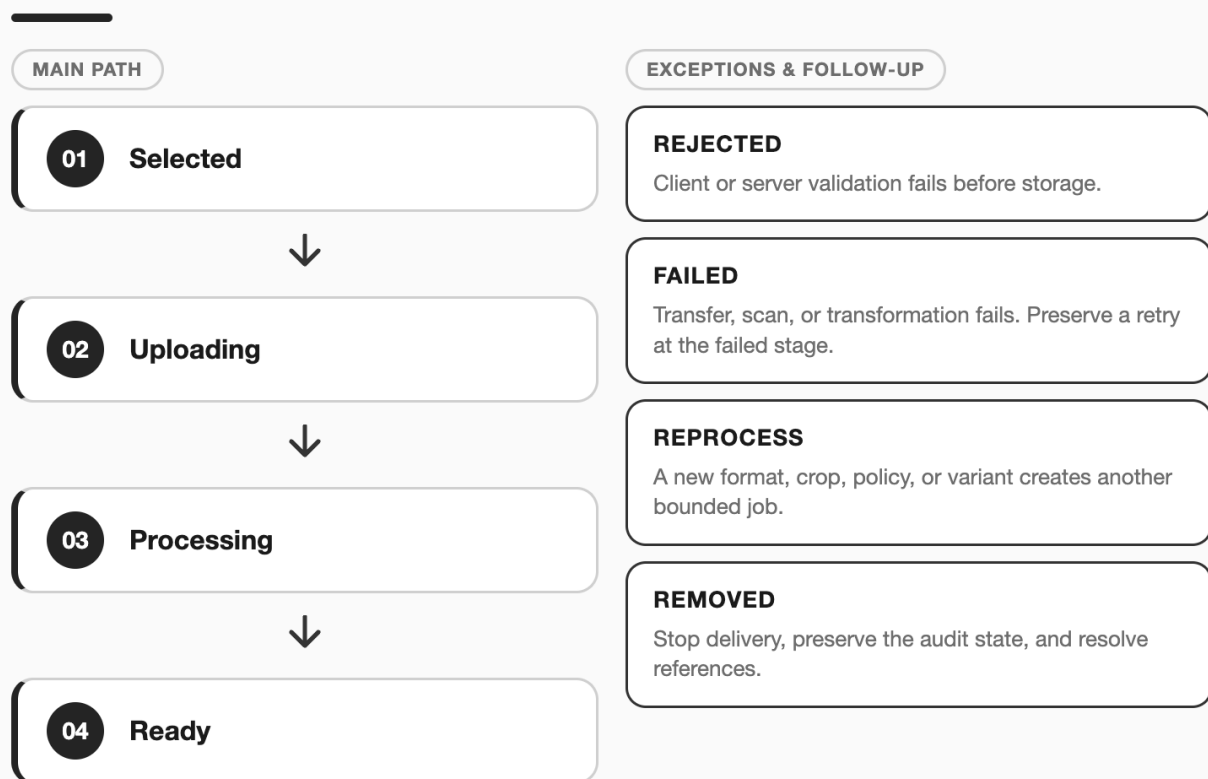
The image-asset journey connects selection and validation to authorised transfer, processing, product state, variants, and delivery. Later sections examine each boundary.

Figure 4.2. Asset-state lifecycle

FIGURE 4.2

The asset state map

“Uploaded” is not “ready.” Validation, transfer, processing, replacement, and removal are visible product states.



Every terminal-looking state still needs ownership and a user-visible next action.

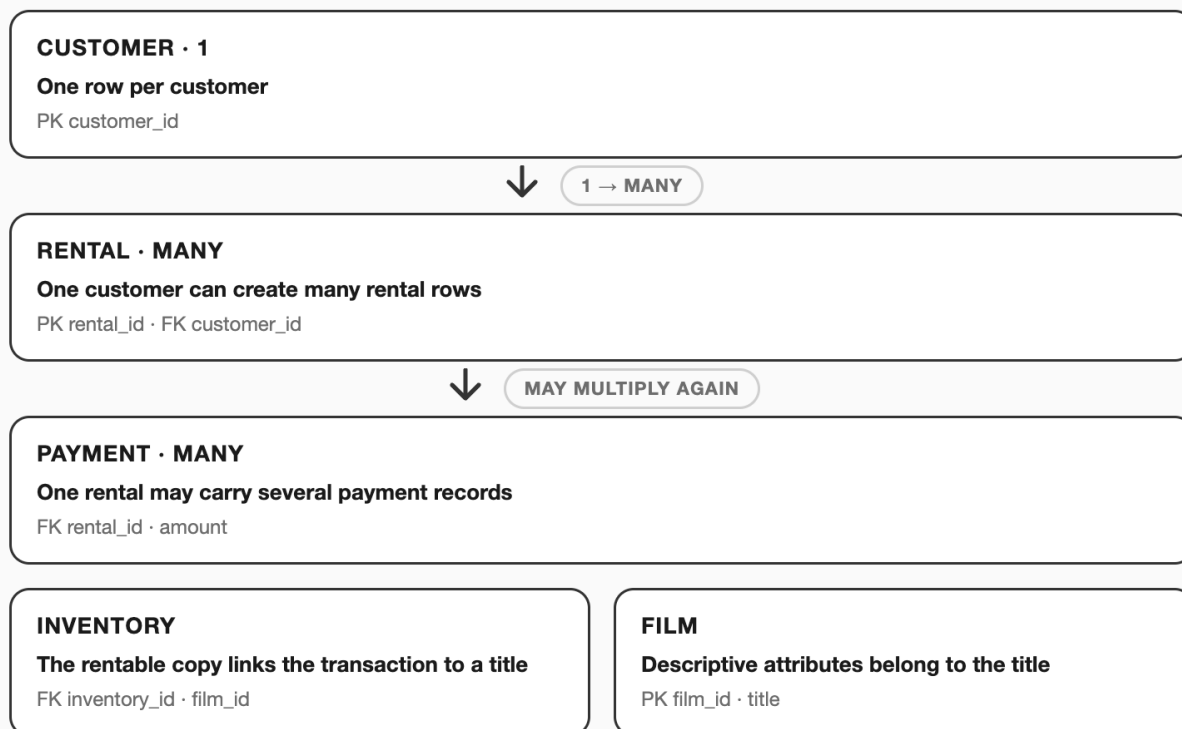
F4.2

Figure 5.1. Relational grain through joins

FIGURE 5.1

How join grain expands

A correct join can still answer the wrong question when one business object becomes many rows.



NAME THE OUTPUT GRAIN BEFORE WRITING THE JOIN

Keys connect tables; the product question decides which row is the unit of analysis.

F5.1

The query grain changes as tables join: one customer can have many rentals, and one rental may have multiple payment records. Confirm the intended unit before aggregating.

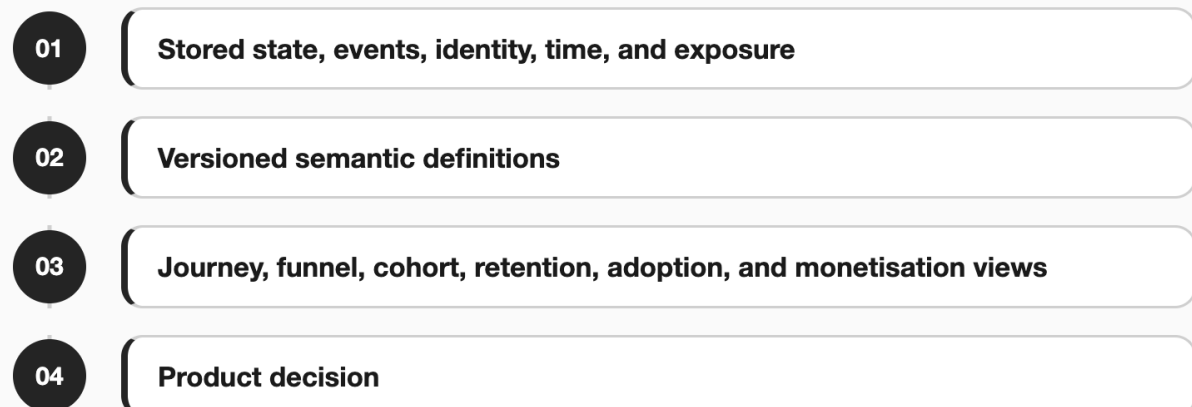
Figure 6.1. Event-to-metric semantic chain

FIGURE 6.1

From product reality to a decision

Analytics is a governed interpretation layer, not an independent dashboard.

—



Every metric inherits the choices made beneath it.

F6.1

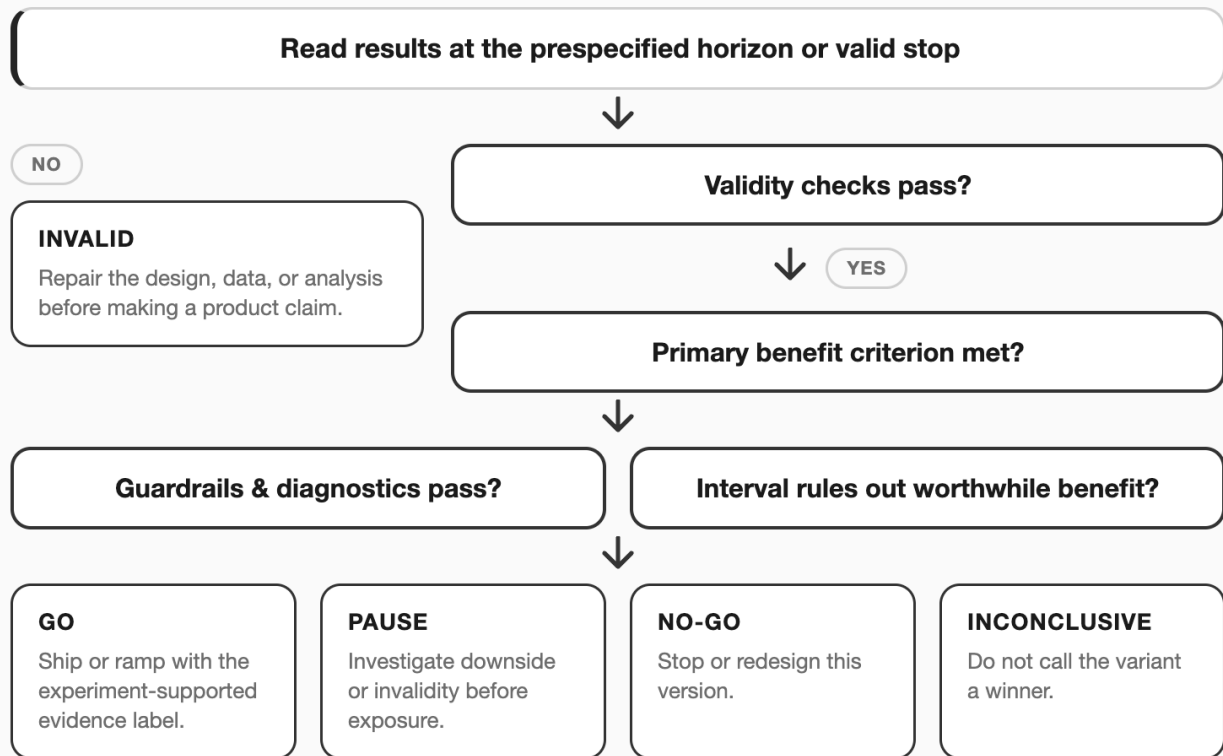
Product analytics combines stored state, event streams, identity, time, and semantic definitions; no dashboard exists independently of those choices.

Figure 7.1. Experiment go/no-go decision flow

FIGURE 7.1

The experiment decision canvas

Evidence validity, product benefit, downside, and action are separate judgments.



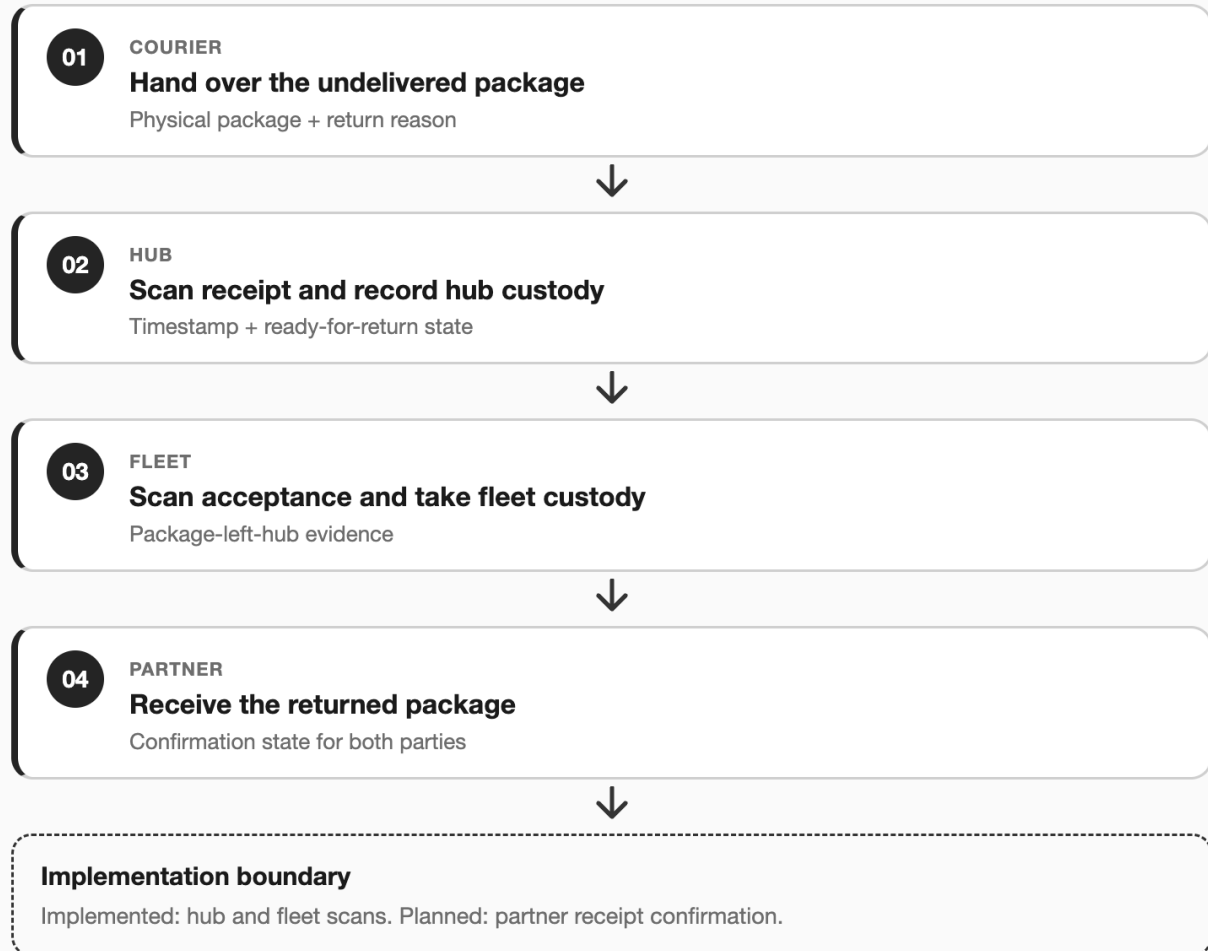
A cautious rollout may still be a product decision, but the evidence label remains inconclusive. F7.1

Figure 8.1. Shipper reverse-logistics custody sequence

FIGURE 8.1

The reverse-logistics custody trail

A return becomes manageable when each physical hand-off leaves a timestamped system receipt.



The backend does not move the package; it makes custody observable and disputable.

F8.1

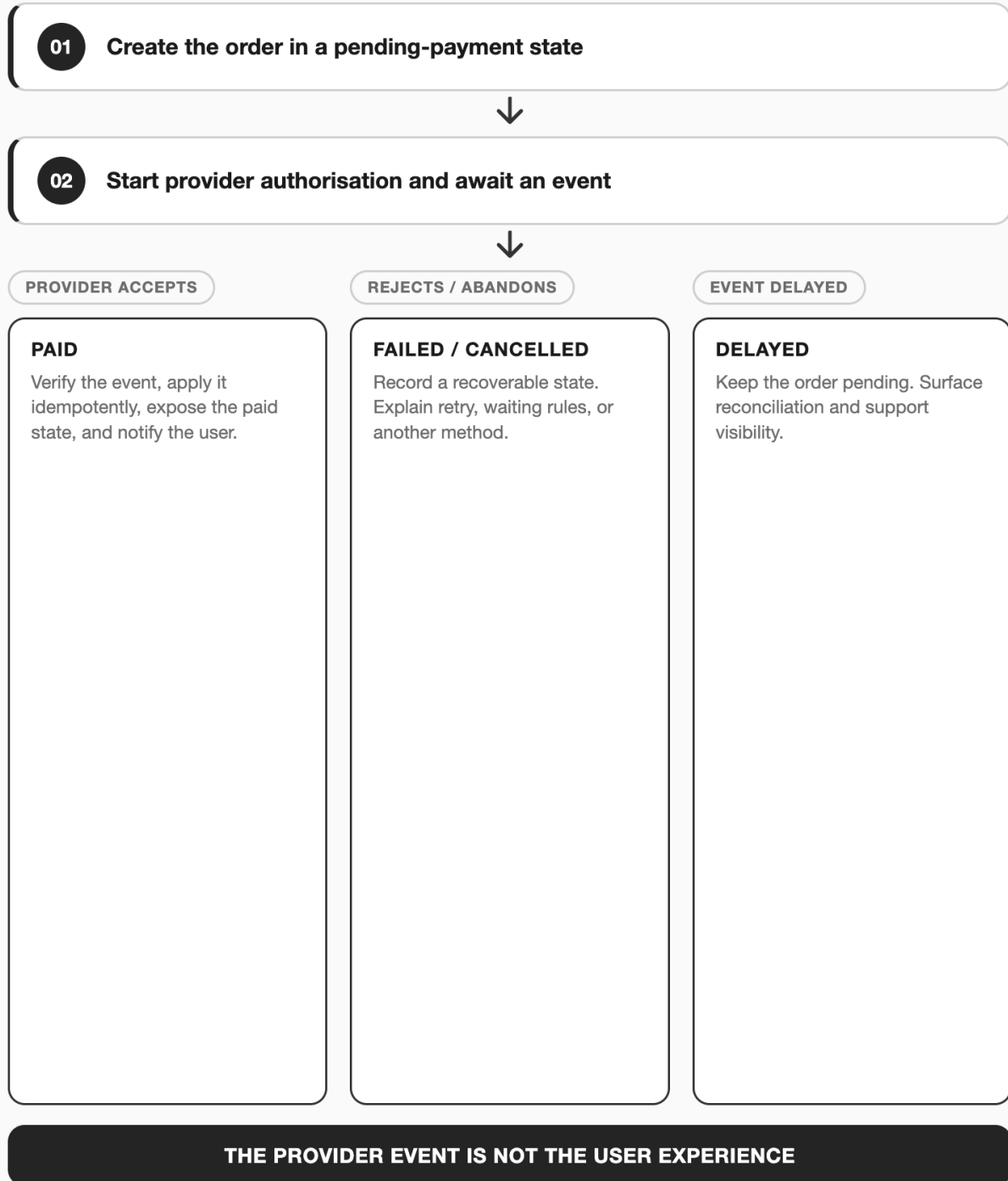
The reverse-logistics flow: hub and driver scanning were implemented, while partner-side receipt confirmation remained planned.

Figure 8.2. E-wallet flow with happy, error, and recovery states

FIGURE 8.2

The e-wallet outcome map

A payment requirement is incomplete until paid, failed, cancelled, and delayed states have product behaviour.



UI, order state, notification, operations, and support must agree on the same outcome.

F8.2

The e-wallet flow makes pending, failed, duplicate-safe, and support-visible behavior part of the requirement rather than late implementation detail.

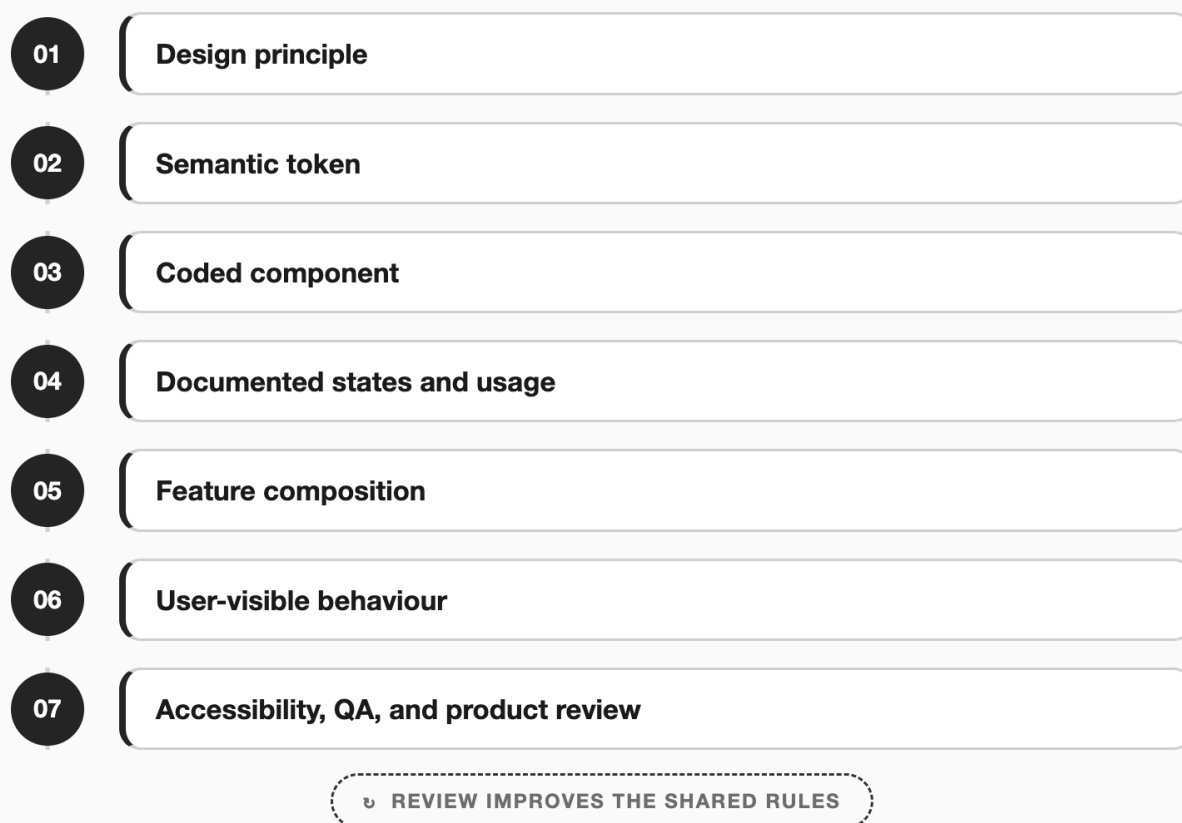
Figure 9.1. Token-component-pattern-product hierarchy

FIGURE 9.1

The design-system value chain

Shared meaning survives only when each layer translates it deliberately.

—



A design decision becomes infrastructure when meaning survives into use.

F9.1

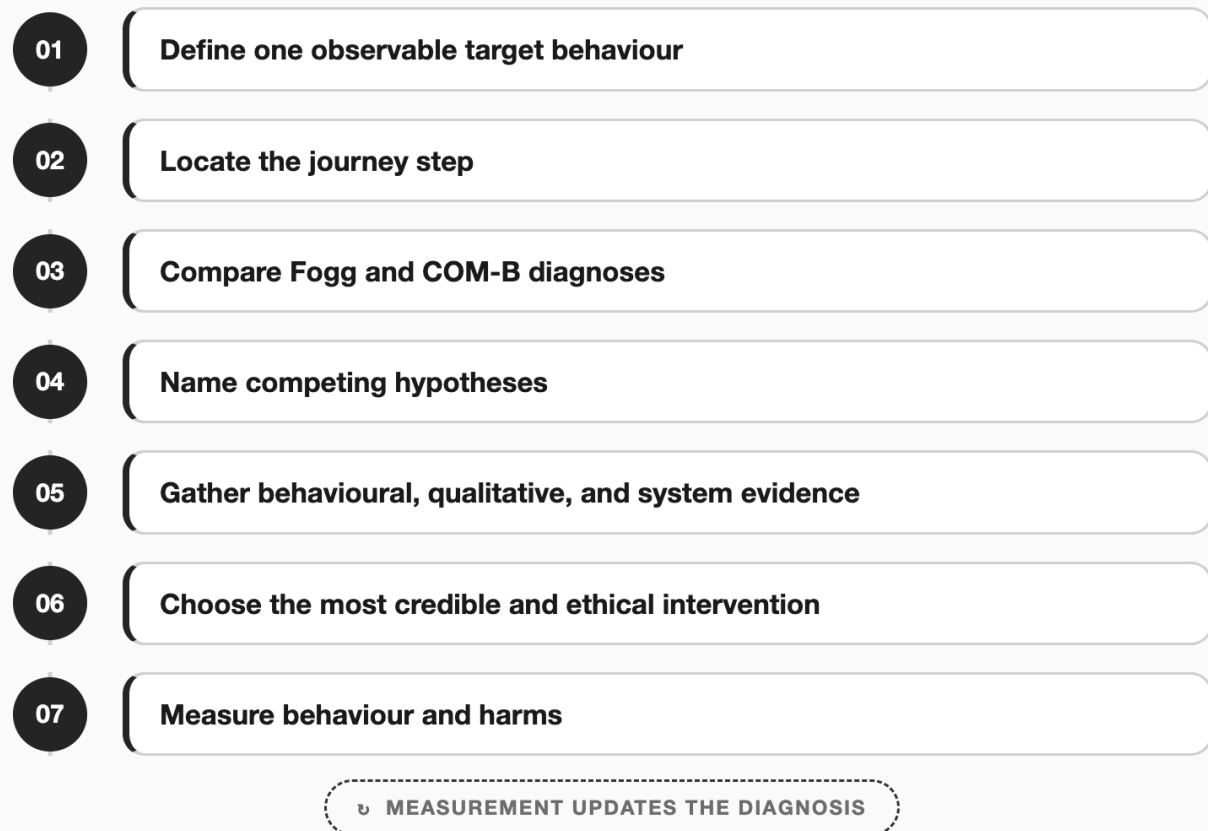
A design decision becomes infrastructure only when meaning survives from principle and token through code, states, composition, and review.

FIGURE 10.1

The behavioural diagnosis loop

Models produce competing explanations; evidence earns the intervention.

—



A plausible behavioural story is a hypothesis until evidence separates it from alternatives.

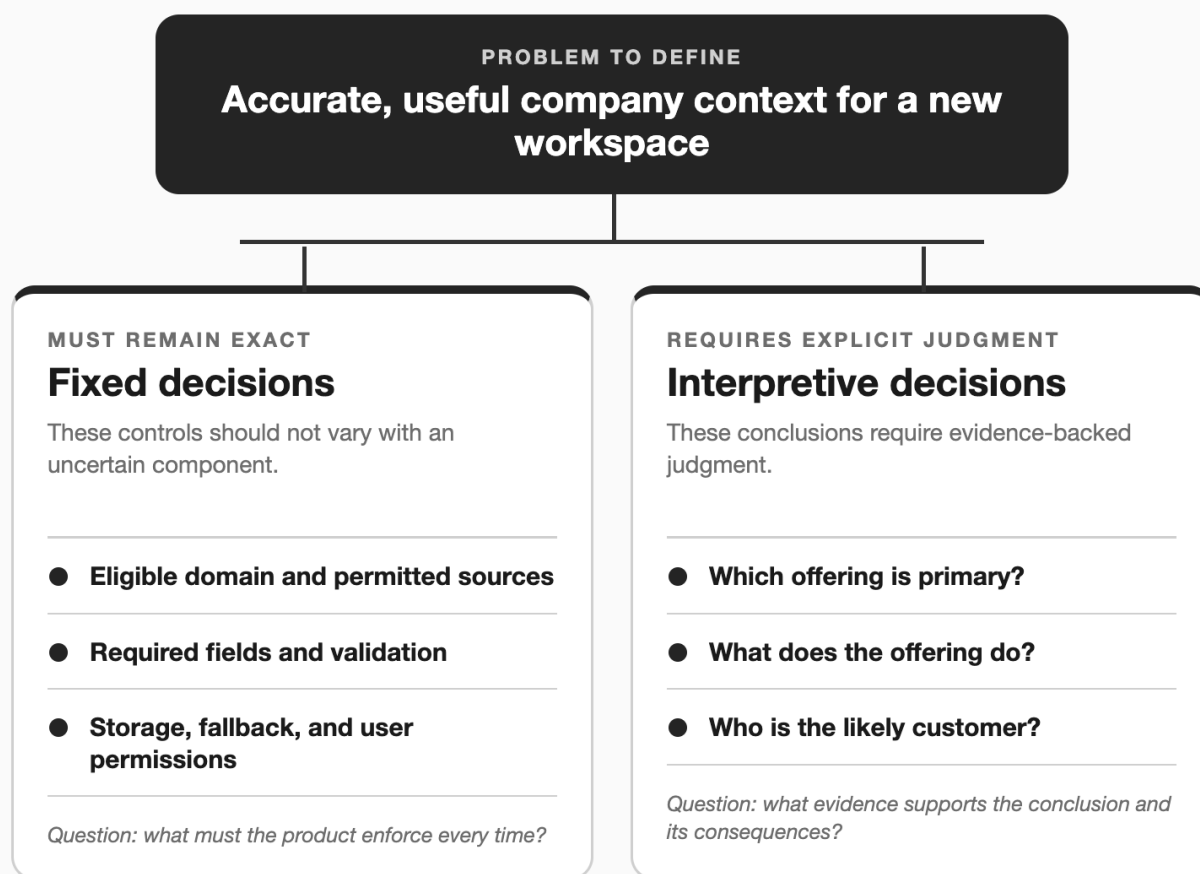
F10.1

Figure 11.1. Onboarding context decision boundary

FIGURE 11.1

The onboarding decision boundary

Define what must be exact and where judgment is required before choosing a solution.



The boundary defines the problem without assigning either side to a specific technology.

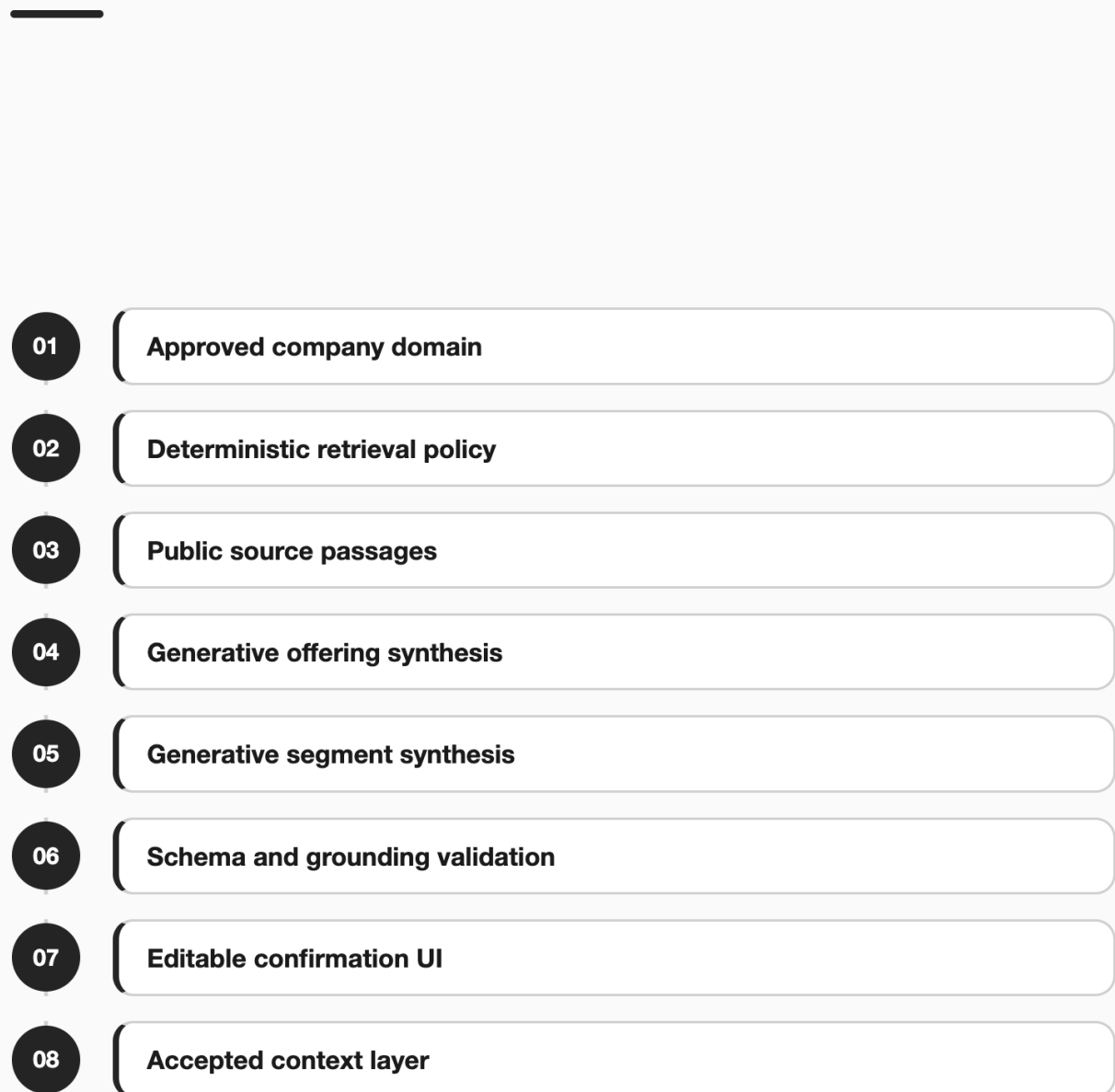
F11.1

Separating fixed decisions from interpretive ones defines the problem boundary without choosing the implementation in advance.

FIGURE 12.1

The hybrid AI responsibility map

Exact control, bounded generation, validation, and user authority do different jobs.



The uncertain component is bounded by controls before and after it.

F12.1

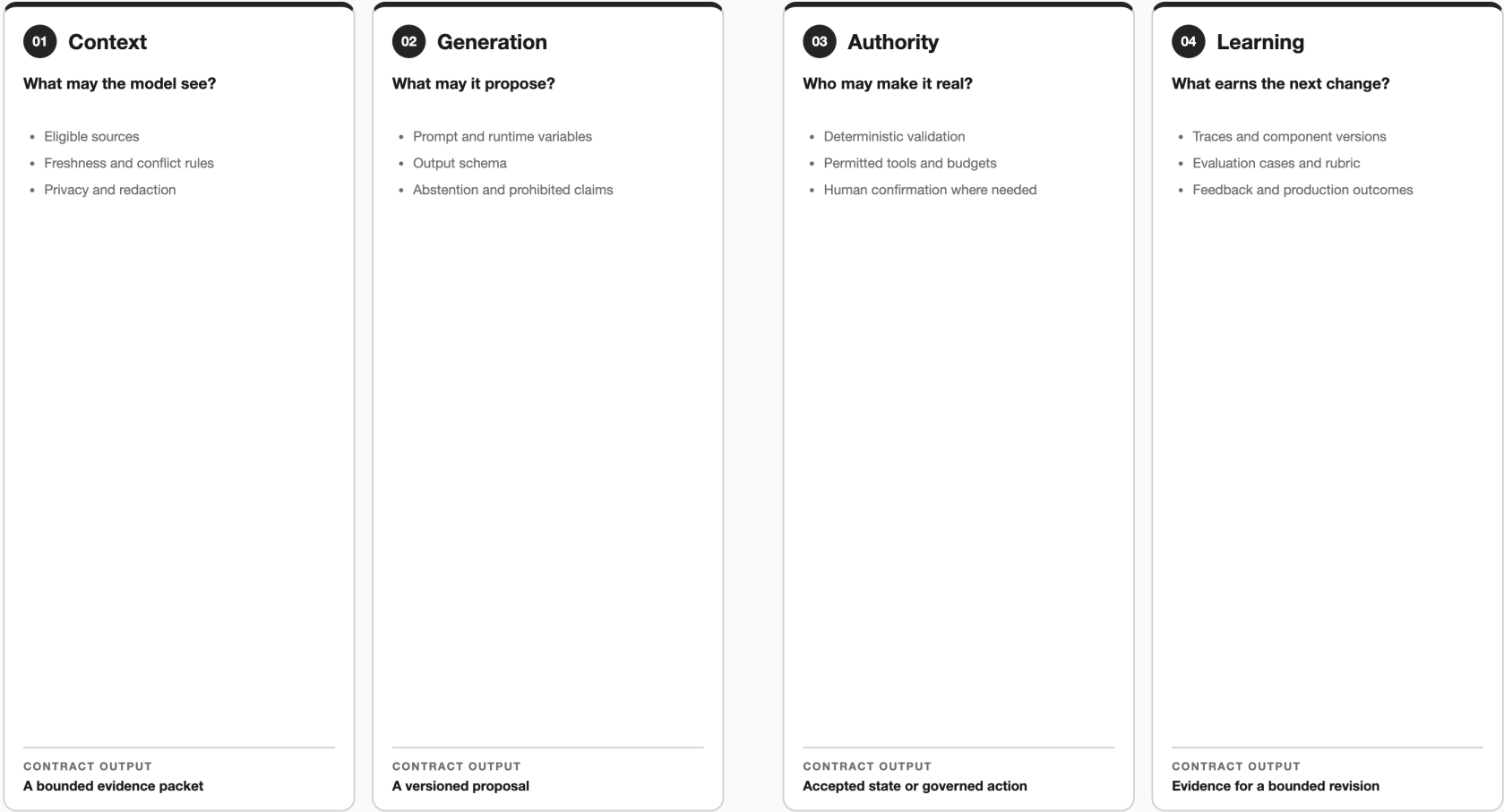
Figure 12.2. Four production contracts around a model

FIGURE 12.2

Four production contracts around a model

The model is one uncertain component. Product usefulness and safety come from explicit contracts around its context, proposal, authority, and learning.

FOUR INSPECTABLE PRODUCTION CONTRACTS



Learning evidence revises context, generation, or authority through a bounded, versioned change.

The contracts can evolve separately, but every production change needs evidence and an accountable owner. F12.2

A production model feature is governed through four contracts: what context is eligible, what the model may return, who may authorise action, and how evidence changes the system.

Figure 13.1. Competitor-ad evaluation pipeline

FIGURE 13.1

The inspectable AI evaluation system

Quality is produced across identity, evidence, analysis, ranking, synthesis, and user trust.

01 DISCOVER



02 COLLECT



03 INTERPRET



04 EARN TRUST



The final answer is only as trustworthy as the evidence path behind it.

Quality is produced across identity, evidence, analysis, ranking, final interpretation, and user trust. **F13.1**

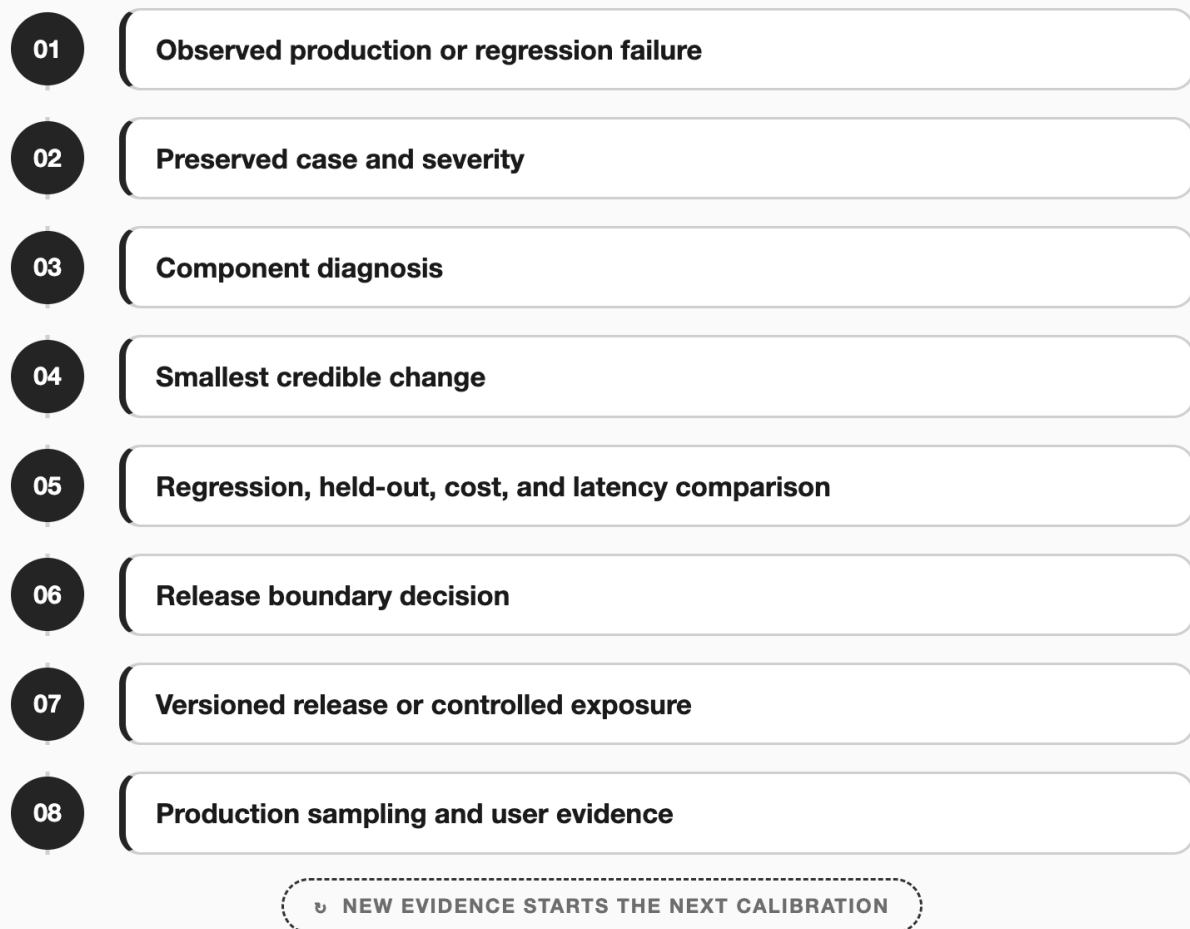
Final-answer quality depends on identity, evidence, analysis, ranking, final interpretation, and UI trust—not only on the generation model.

Figure 14.1. Human-directed Heatseeker calibration loop

FIGURE 14.1

The calibration evidence loop

A production failure becomes a preserved case, bounded change, and measured release.



The team learns from governed cases; the model does not supervise itself.

F14.1

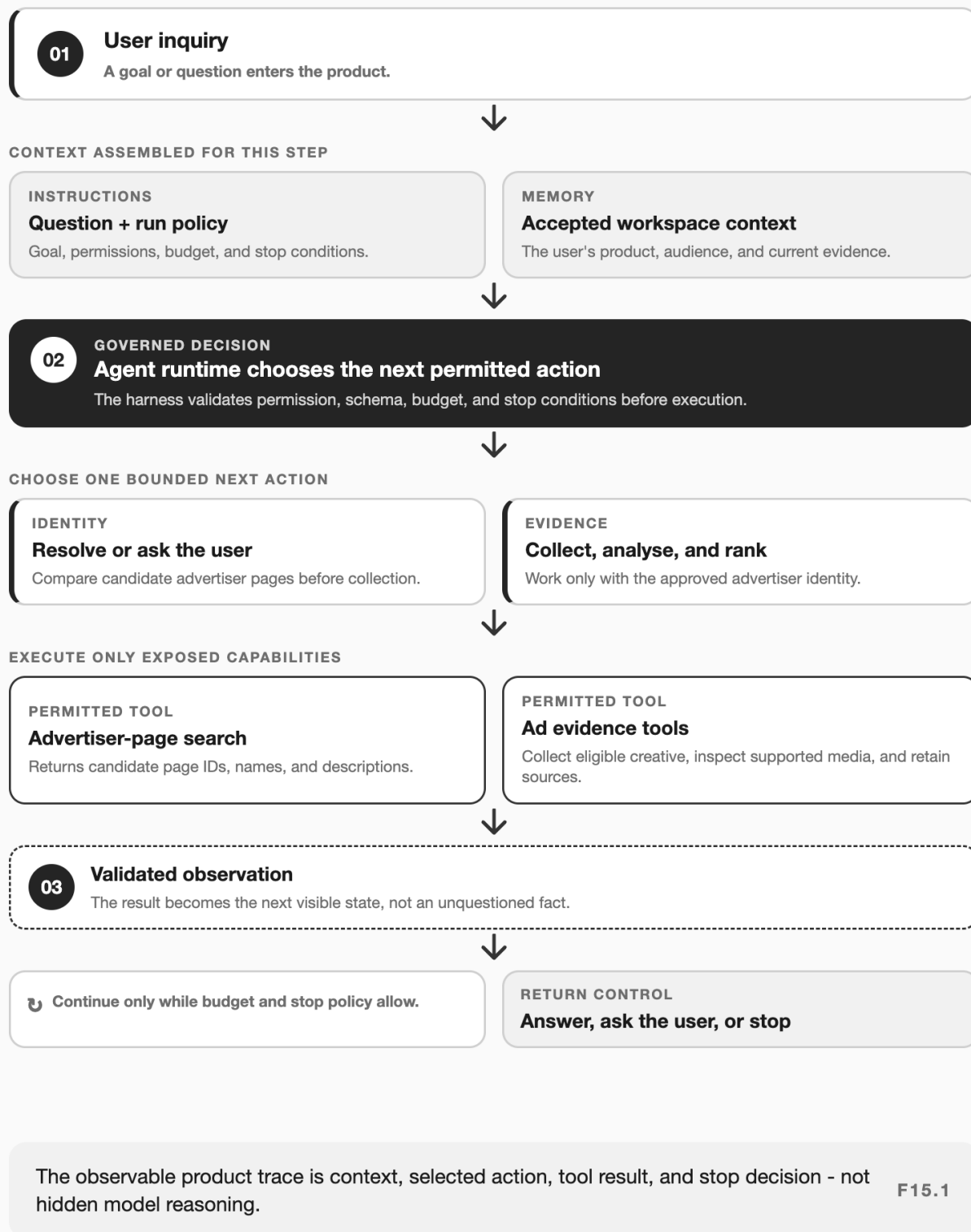
Calibration is a human-directed quality loop: an observed failure becomes a preserved case, bounded change, regression-tested release, and new production evidence.

Figure 15.1. Bounded competitor-analysis loop

FIGURE 15.1

The bounded competitor-analysis loop

One run moves through approved context, permitted actions, validated evidence, and explicit stop decisions.



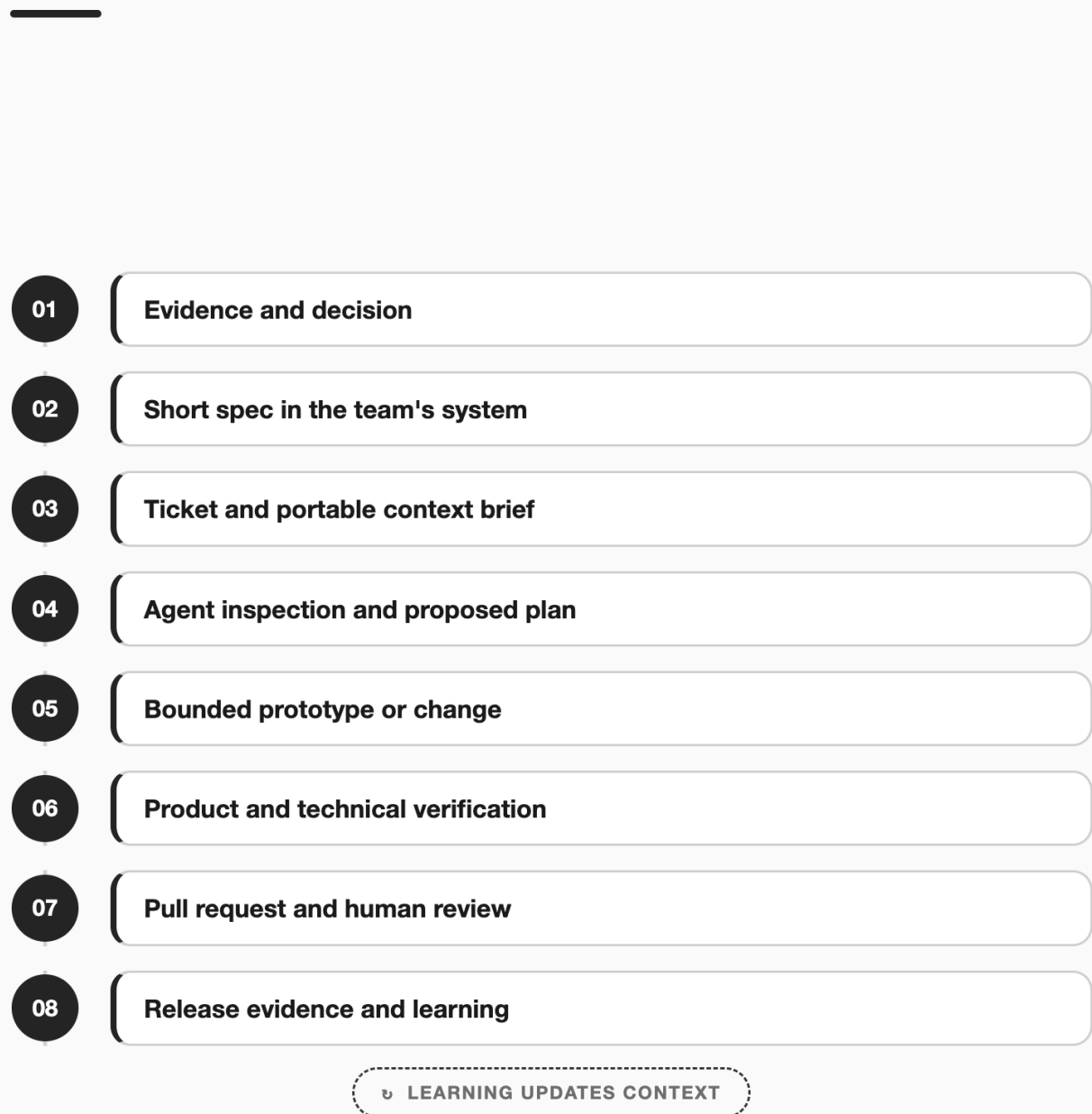
The observable competitor-analysis loop: the application assembles governed context, permits a bounded next action, validates the result, and either continues within its limits or returns control to the user.

Figure 16.1. Agent operating contract

FIGURE 16.1

From evidence to verifiable work

Portable context keeps product intent intact across agents, artifacts, and review.



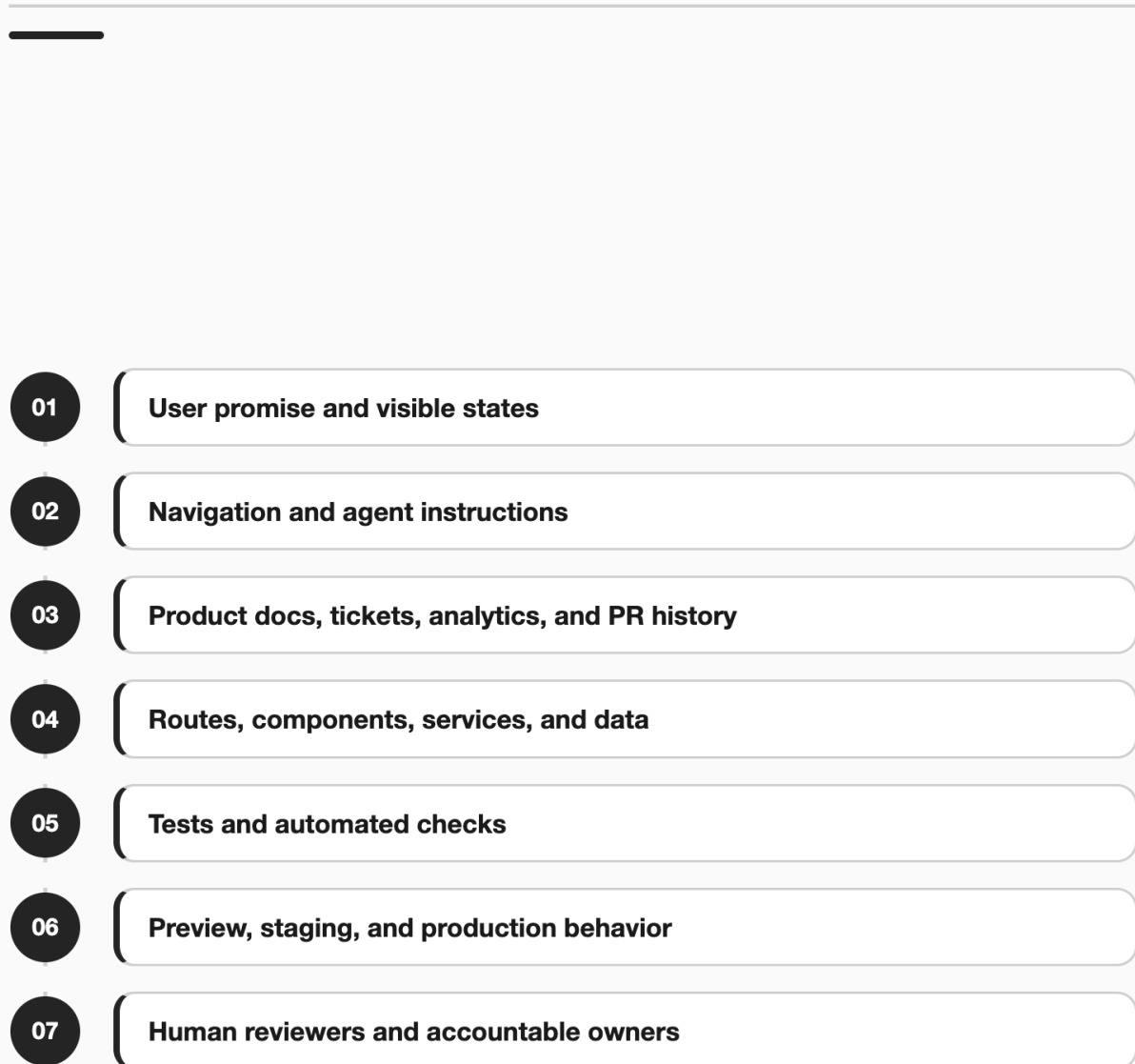
The useful unit of agent work is a verifiable contribution, not a prompt transcript.

F16.1

FIGURE 17.1

The repository orientation path

Start from the promise, then trace through instructions, implementation, checks, and ownership.



Product context explains intent; runtime evidence and accountable owners verify reality.

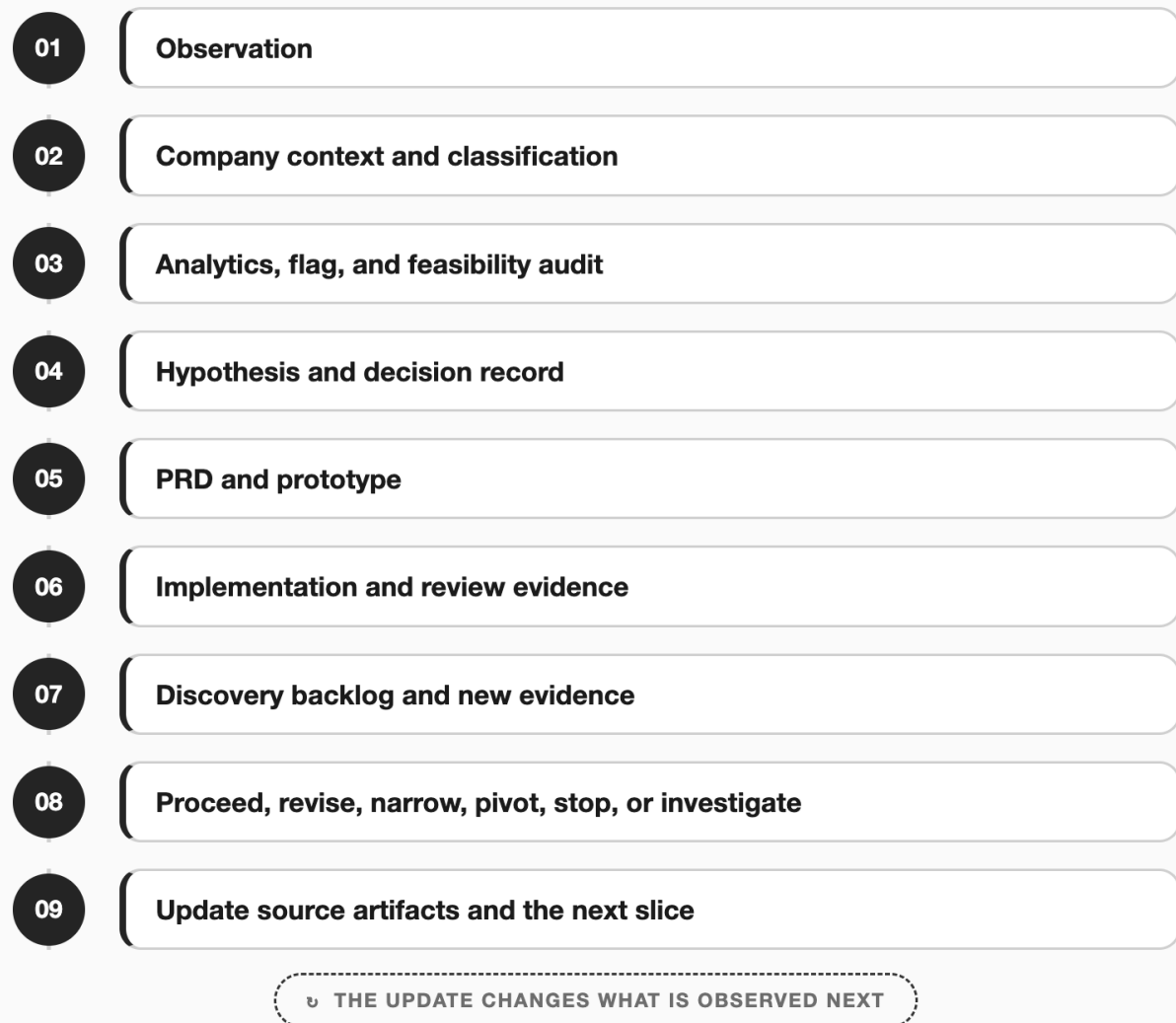
F17.1

Figure 18.1. Evidence operations continuity

FIGURE 18.1

The evidence operations chain

Each artifact inherits a decision thread and leaves a visible update for the next one.



Evidence lineage is preserved when every transition shows what it inherited and changed.

F18.1

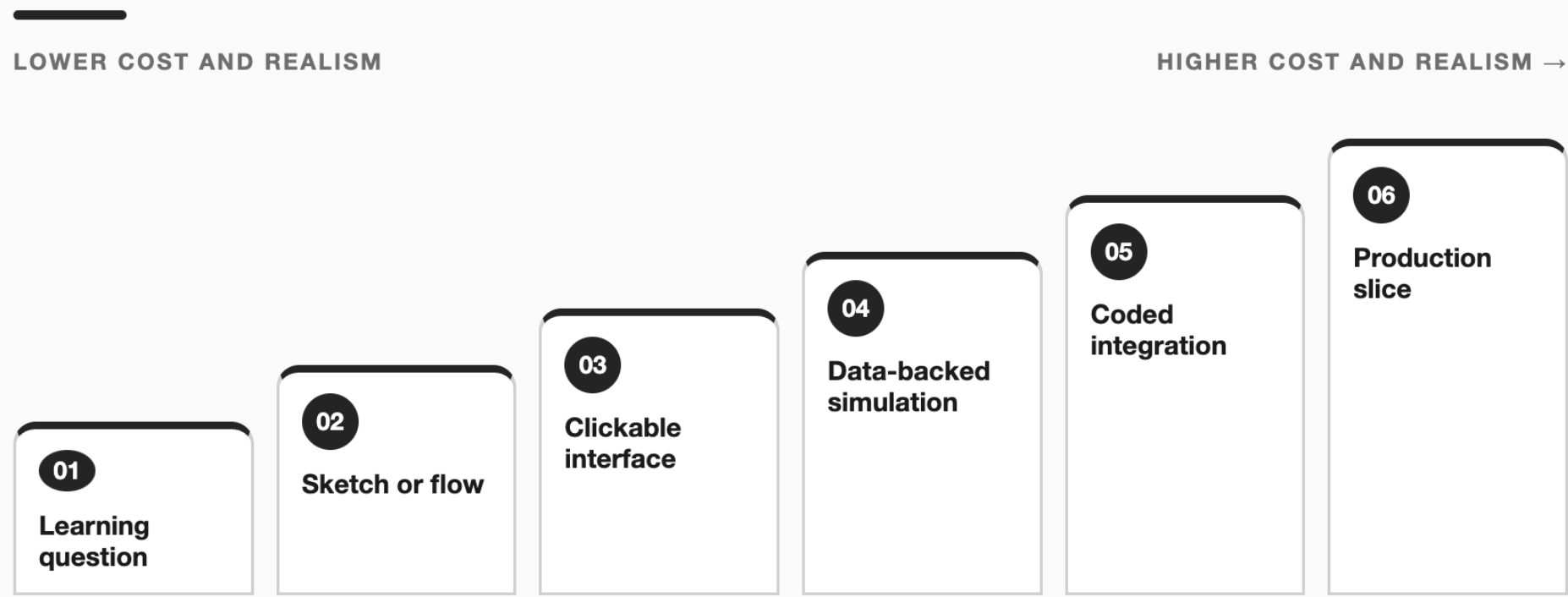
Evidence operations preserve lineage by carrying each observation through classification, product artifacts, implementation evidence, and an explicit decision that updates the next source of truth.

Figure 19.1. Prototype fidelity ladder

FIGURE 19.1

The prototype fidelity ladder

Increase fidelity only when the learning question demands a more expensive proof.



Prototype fidelity is a decision ladder, not a maturity race.

F19.1

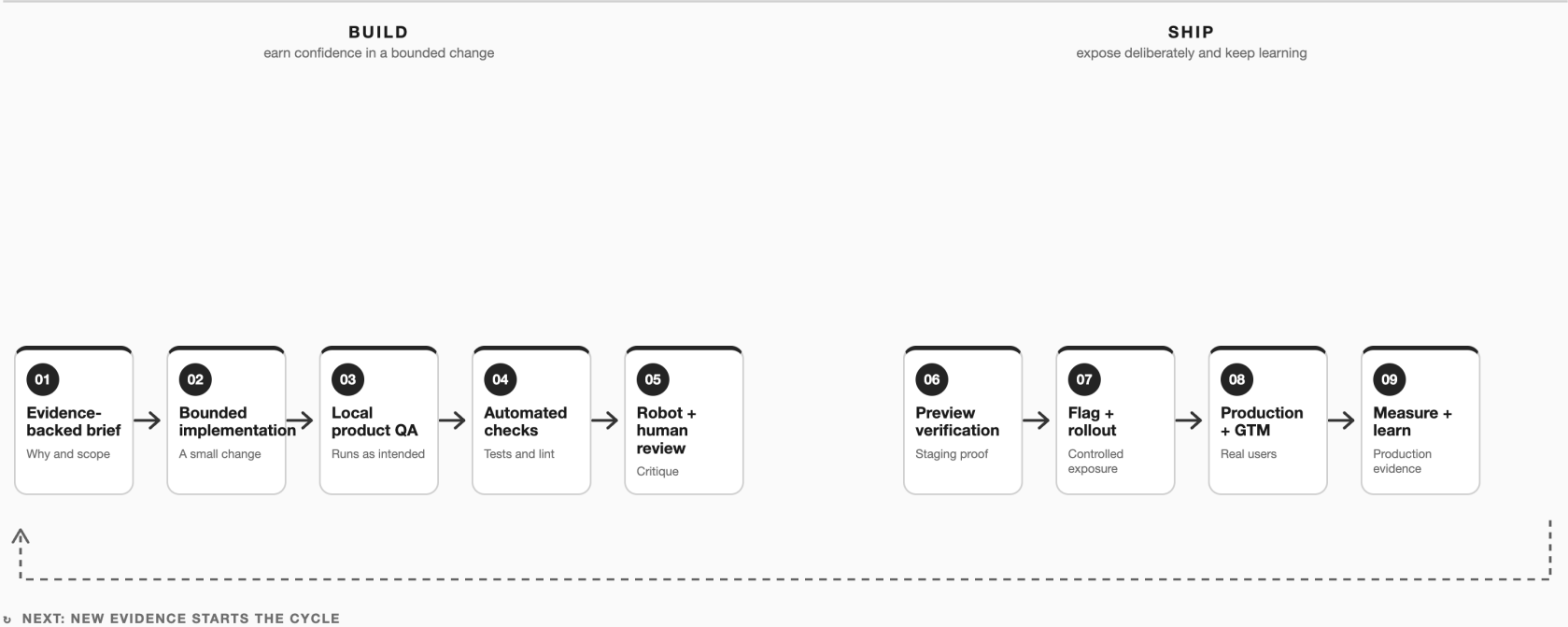
Prototype fidelity is a decision ladder, not a maturity race: move only as far as the learning question requires, and treat production exposure as a separate commitment.

Figure 20.1. Building is not shipping

FIGURE 20.1

Building is not shipping

A change becomes a product only after it survives verification, review, controlled exposure, and learning.



Local confidence is necessary; observable production behaviour completes the work. F20.1

Building becomes shipping only after the change survives verification, review, controlled exposure, and measurement.